R Code Examples from WDDA Lecture 02

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-04-29

Contents

1 Introduction

This document collects all R code examples from the WDDA Lecture 02 slides. The focus is on descriptive statistics, in particular tabular and graphical representations of data. The data used here can be found in `data/WDDA_02.xlsx`.

Descriptive statistics is a fundamental area of statistics that deals with the summary, visualization, and interpretation of data. In contrast to inferential statistics, which draws conclusions about a population based on samples, descriptive statistics describes the data at hand without making further inferences.

2 Preparation

```r
# Load the required packages
library(readxl)  # Package for importing Excel files

# Load the datasets
# (Auto, variable "Brand")
Brand <- read_excel("../../data/WDDA_02.xlsx", sheet = "Auto")

# (Audit, variable "Time")
Time <- read_excel("../../data/WDDA_02.xlsx", sheet = "Audit")
# Convert Time to numeric vector
# unlist() converts the data structure into a vector
# as.numeric() converts the values into numeric values
Time <- as.numeric(unlist(Time))

# (Restaurant)
Restaurant <- read_excel("../../data/WDDA_02.xlsx", sheet = "Restaurant")

# (Inventory, variables "shirt", "price")
Inventory <- read_excel("../../data/WDDA_02.xlsx", sheet = "Inventory")

# (Stereo, variables "Week", "Commercials", "Sales")
Stereo <- read_excel("../../data/WDDA_02.xlsx", sheet = "Stereo")
```

Note on R packages: In R, additional functionality is provided through packages. The `readxl` package allows you to read Excel files. Packages must first be installed with `install.packages("packagename")` and then loaded with `library(packagename)`.

Note on data paths: Note that the R Markdown files use relative paths (`../data/`), while the R scripts use direct paths (`data/`). This is because R Markdown files work from the location of the file, whereas R scripts work from the working directory.

3 Frequency Distributions

A frequency distribution summarises data in tabular form by indicating the number of items in non-overlapping classes.

```r
# Identify unique values
unique_brands <- Brand |> unique()  # Alternative notation: unique(Brand)

# Count frequencies
freq <- Brand |> table()  # The table() function creates a frequency table
print(freq)  # Output the frequency table
```

```
## Brand
##     Audi      BMW Mercedes     Opel       VW
##        8        5       13        5       19
```

Statistical explanation: A frequency distribution is a fundamental method for summarising data. It shows how often each value or category occurs in a dataset. For categorical data (such as car brands), this is especially useful for getting an overview of the distribution.

R explanation:

- The pipe operator `|>` in R (introduced in R 4.1.0) passes the result of the expression on the left as the first argument to the function on the right.
- The function `unique()` returns all unique values in a vector.
- The function `table()` creates a frequency table that counts the number of occurrences for each unique category.

4 Relative Frequencies

The relative frequency of a category corresponds to the proportion of the absolute frequency relative to the total frequency.

```
# Compute the relative frequencies
relfreq <- freq / sum(freq)
print(relfreq)  # Output the relative frequencies
```

```
## Brand
##     Audi      BMW Mercedes     Opel       VW
##     0.16     0.10     0.26     0.10     0.38
```

```
# Alternative method using prop.table()
relfreq_alt <- prop.table(freq)
print(relfreq_alt)  # Returns the same result as above
```

```
## Brand
##     Audi      BMW Mercedes     Opel       VW
##     0.16     0.10     0.26     0.10     0.38
```

Statistical explanation: Relative frequencies indicate the proportion of each category relative to the total amount and are expressed as decimal numbers or percentages. They are particularly useful for comparisons between datasets of different sizes. The sum of all relative frequencies is always 1 (or 100%).

R explanation:

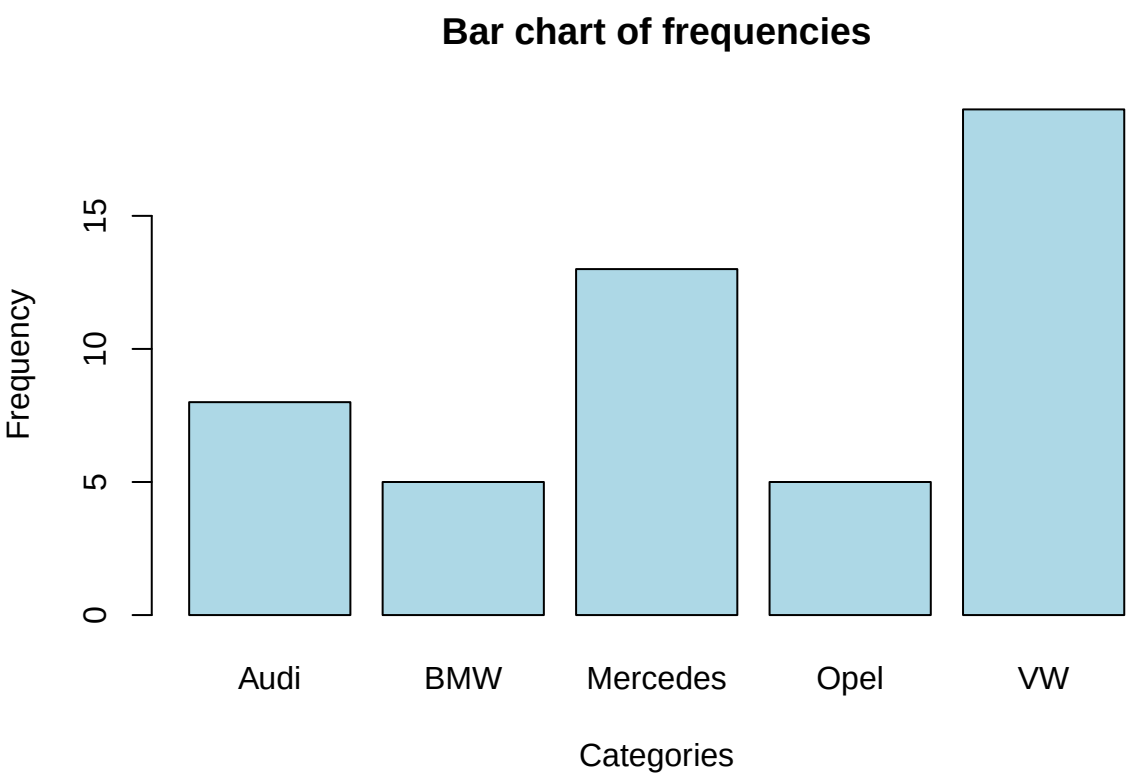
- Dividing a frequency table by its sum (`freq / sum(freq)`) computes the relative frequencies.
- The function `prop.table()` is a specialised function in R that computes relative frequencies directly from a frequency table.
- Both methods yield the same result, but `prop.table()` is often shorter and more readable.

5 Graphical Representations

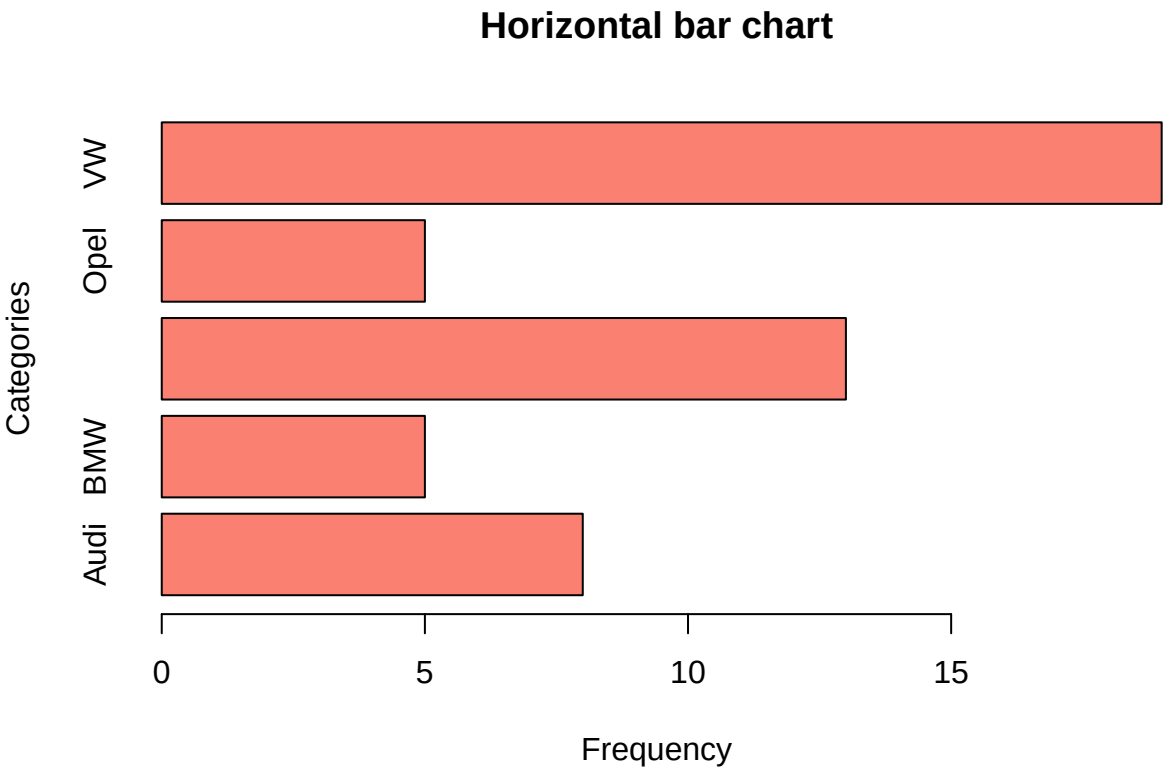
5.1 Bar Charts

Bar charts visualize the absolute frequency of categorical data.

```
# Create a bar chart
barplot(freq,
        main = "Bar chart of frequencies",  # Title of the chart
        col = "lightblue",                  # Color of the bars
        xlab = "Categories",                # x-axis label
        ylab = "Frequency")                 # y-axis label
```




```r
# Create a horizontal bar chart
barplot(freq,
        horiz = TRUE,                          # Horizontal orientation
        main = "Horizontal bar chart",
        col = "salmon",
        xlab = "Frequency",
        ylab = "Categories")
```



Statistical explanation: Bar charts are one of the most common methods for visualizing categorical data. Each bar represents a category, and the height (or length for horizontal charts) corresponds to the frequency. Bar charts are particularly well suited for nominal data (categories without a natural ordering) and ordinal data (categories with a natural ordering).

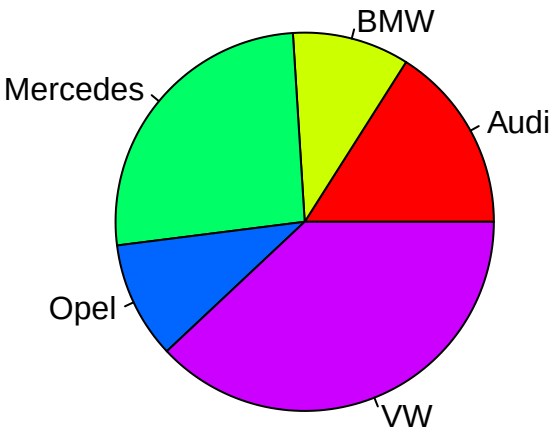
R explanation:

- The function `barplot()` creates a bar chart from a frequency table or a numeric vector.
- With the parameter `horiz = TRUE` you can create a horizontal bar chart.
- Other important parameters are:
  - `main`: title of the chart
  - `col`: color of the bars (R supports many color names and RGB values)
  - `xlab` and `ylab`: axis labels
  - `names.arg`: alternative labels for the categories

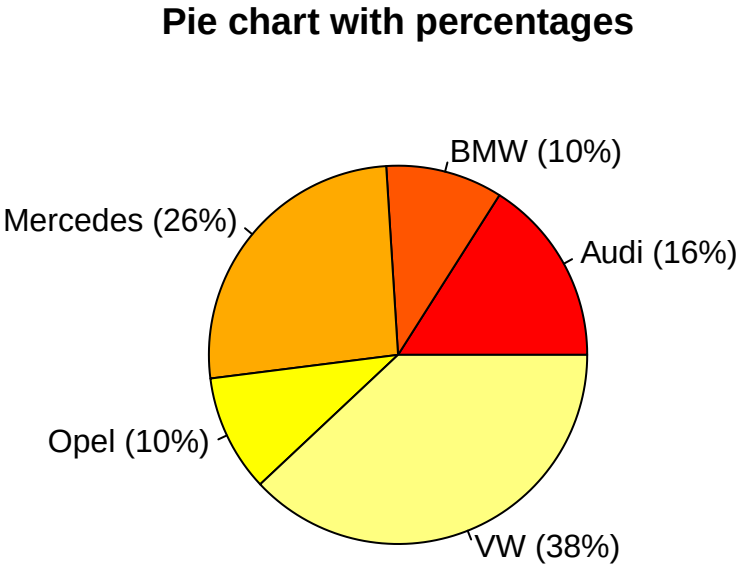
5.2 Pie Charts

Pie charts visualize the relative frequency of nominal data.

```r
# Create a pie chart
pie(freq,
    main = "Pie chart of relative frequencies",
    col = rainbow(length(freq)))  # Use different colors
```




```r
# Pie chart with percentages
pie(freq,
    main = "Pie chart with percentages",
    labels = paste0(names(freq), " (", round(100*relfreq, 1), "%)"),
    col = heat.colors(length(freq)))
```



Statistical explanation: Pie charts represent the relative shares of different categories as sectors of a circle. The size of each sector is proportional to the relative share of the corresponding category. Pie charts are best suited for nominal data with few categories (ideally no more than 5–7), since too many sectors can impair readability.

R explanation:

- The function `pie()` creates a pie chart from a numeric vector.
- With `rainbow()`, `heat.colors()`, and similar functions, color palettes can be created automatically.
- The parameter `labels` allows custom labels for the sectors.
- The function `paste0()` concatenates strings without a separator, while `paste()` inserts spaces by

default.

Note on visualization: Although pie charts are widespread, they are often viewed critically in professional data analysis because humans find it harder to compare angles and areas than lengths. Bar charts are therefore often the better choice for precise comparisons.

6 Summarising Quantitative Data

For quantitative data with many unique values, grouping into classes (bins) is useful.

```
# Frequency table for ungrouped data
table(Time)
```

```
## Time
## 12 13 14 15 16 17 18 19 20 21 22 23 27 28 33
##  1  1  2  2  1  1  3  1  1  1  2  1  1  1  1
```

```
# Define classes (bins)
bins <- c(0, 14 + 5 * (0:5))  # Extends to 0, 14, 19, 24, 29, 34, 39
print(bins)  # Show the defined class boundaries
```

```
## [1]  0 14 19 24 29 34 39
```

```
# Group the data
Time_binned <- Time |> cut(bins)
table(Time_binned)
```

```
## Time_binned
##  (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##       4       8       5       2       1       0
```

Statistical explanation: For continuous or discrete data with many different values, it is often useful to group the data into classes (bins). This simplifies the presentation and interpretation of the data. The choice of the number and width of classes is an important step that can influence the informativeness of the analysis:

- Too few classes can hide important patterns
- Too many classes can lead to a fragmented presentation that is difficult to interpret

A rule of thumb is Sturges' rule: k 1 + 3.322 × log10(n), where k is the number of classes and n is the sample size.

R explanation:

- The function `cut()` divides numeric data into intervals (bins).
- The first parameter is the vector to be grouped.
- The second parameter specifies the boundaries of the intervals.
- The result is a factor whose levels represent the intervals.
- The notation `(a,b]` means that the interval excludes a and includes b.

7 Cumulative Frequencies

The cumulative frequency indicates how many data values are less than or equal to the upper bound of the respective class.

```r
# Total sum of values in 'Time' (only as an example, not always meaningful)
sum(Time)
```

```
## [1] 385
```

```r
# Create the cumulative frequency table
cum_freq <- Time_binned |>
  table() |>
  cumsum()
print(cum_freq)  # Output the cumulative frequencies
```

```
##  (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##       4      12      17      19      20      20
```

```r
# Compute the cumulative relative frequencies
cum_rel_freq <- cum_freq / sum(table(Time_binned))
print(cum_rel_freq)  # Output the cumulative relative frequencies
```

```
##  (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##    0.20    0.60    0.85    0.95    1.00    1.00
```

Statistical explanation: Cumulative frequencies sum the frequencies up to a certain point. They are useful for determining how many observations lie below a particular value. Cumulative relative frequencies express this share as a proportion (between 0 and 1).

Cumulative frequencies are often used for:

- Computing percentiles and quartiles
- Constructing cumulative distribution functions
- Analysing thresholds in data

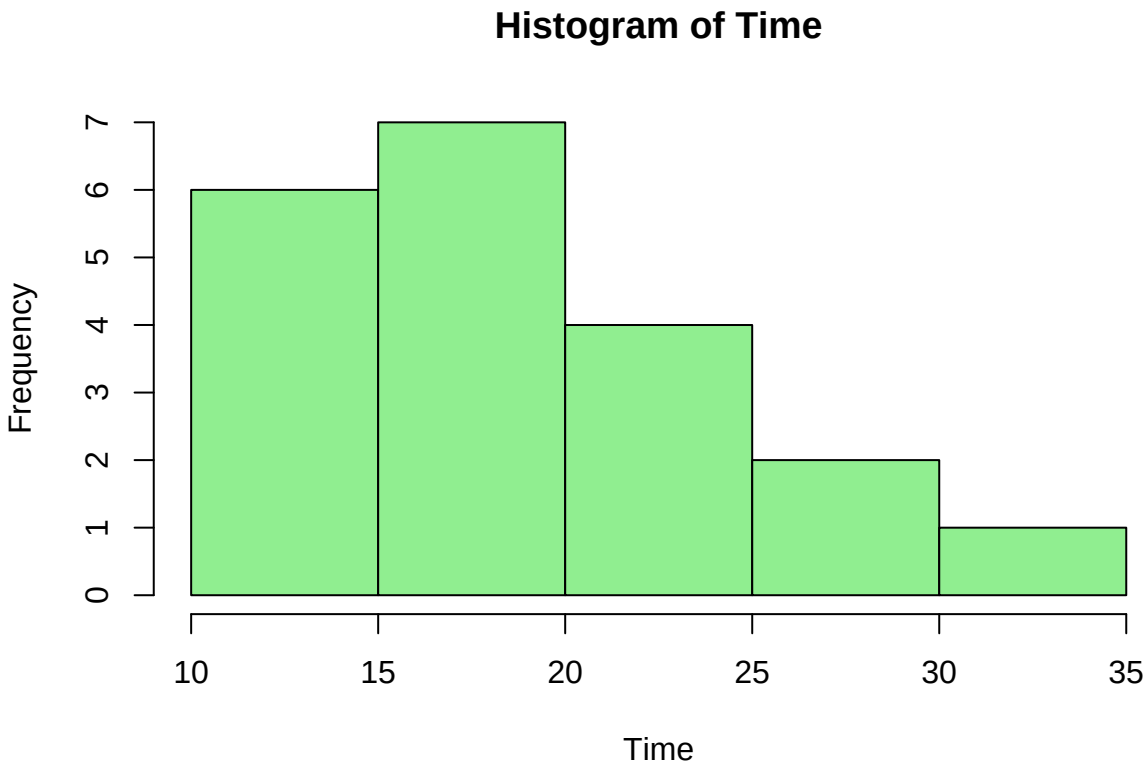
R explanation:

- The function `cumsum()` computes the cumulative sum of a vector.
- In the pipe chain, a frequency table is created first and then the cumulative sum is computed.
- Dividing by the total sum yields the cumulative relative frequencies.

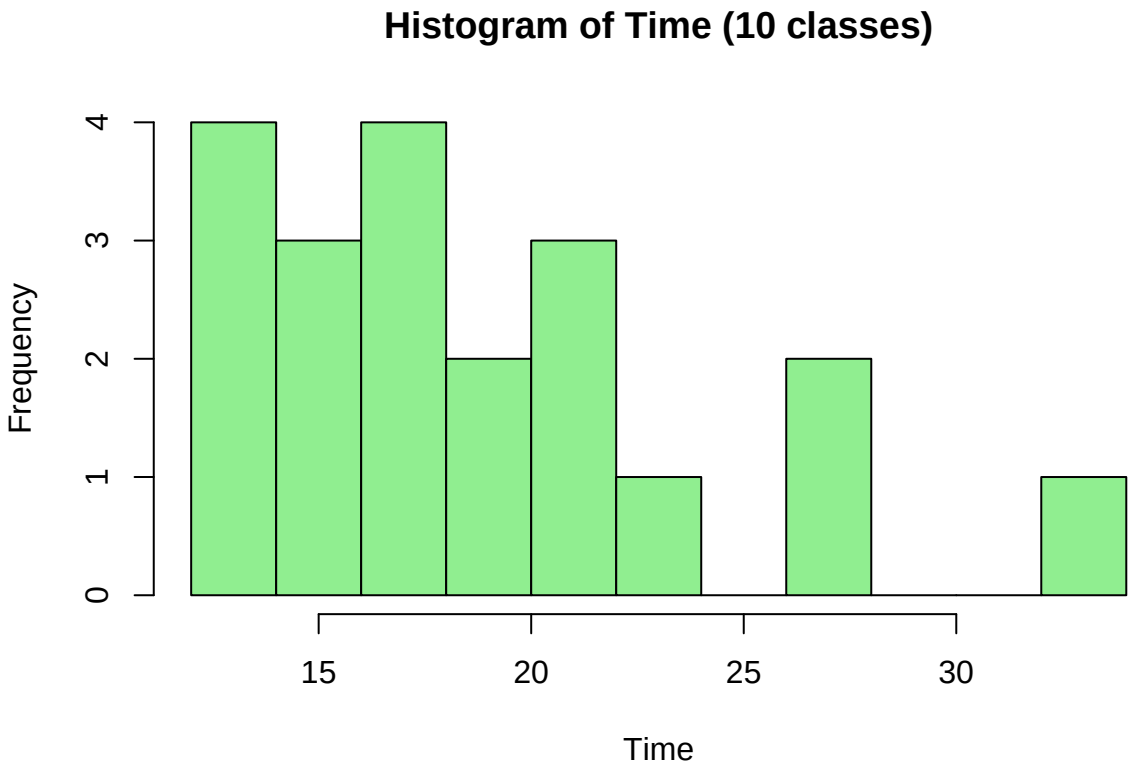
8 Histograms

Histograms are the most important graphical representation for quantitative data.

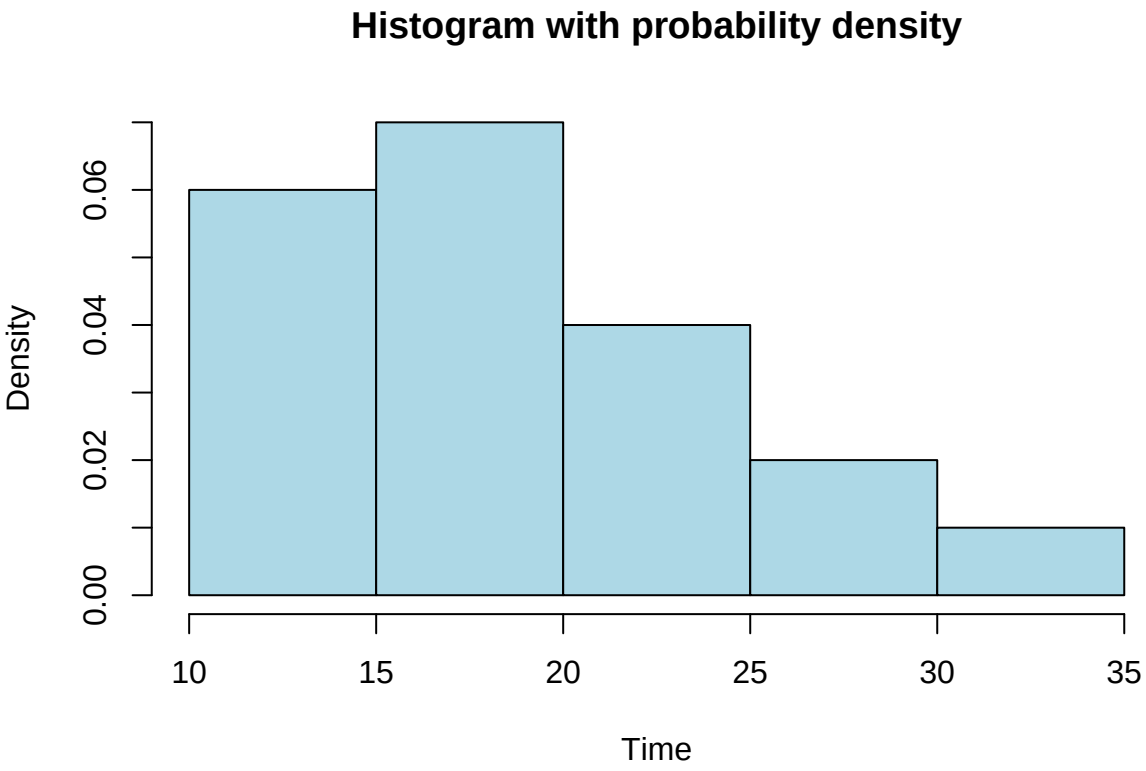
```r
# Standard histogram
hist(Time,
     main = "Histogram of Time",
     xlab = "Time",
     col = "lightgreen")
```




```
# Histogram with adjusted number of classes
hist(Time,
     breaks = 10,
     main = "Histogram of Time (10 classes)",
     xlab = "Time",
     col = "lightgreen")
```




```r
# Histogram with probability density instead of frequencies
hist(Time,
     freq = FALSE,  # Shows density instead of frequencies
     main = "Histogram with probability density",
     xlab = "Time",
     col = "lightblue")
```



Statistical explanation: Histograms are a fundamental graphical representation for continuous data. They divide the range of values into intervals (bins) and show the frequency or density of observations in each interval. Histograms help to:

- Recognize the shape of the distribution (symmetric, right-skewed, left-skewed, bimodal, etc.)
- Identify outliers
- Visually grasp the spread and central tendency of the data

The choice of the number of classes is crucial: too few classes can hide important patterns, too many can lead to a noisy representation.

R explanation:

- The function `hist()` creates a histogram from a numeric vector.
- The parameter `breaks` controls the number or position of the class intervals:
  - A single number indicates the approximate number of desired classes
  - A vector specifies the exact class boundaries
- With `freq = FALSE`, the probability density is shown instead of the absolute frequency.
- The area under a density histogram sums to 1.

Tip: When choosing the number of classes, the goal is to model the signal (pattern) and not the noise. Experiment with different values for `breaks` to find the most informative representation.

9 Skewness

Histograms can give clues about the skewness (asymmetry) of the distribution.

```
# Compute the skewness with the 'e1071' package
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
```

```r
library(e1071)
skewness_value <- skewness(Time)
print(skewness_value)
```

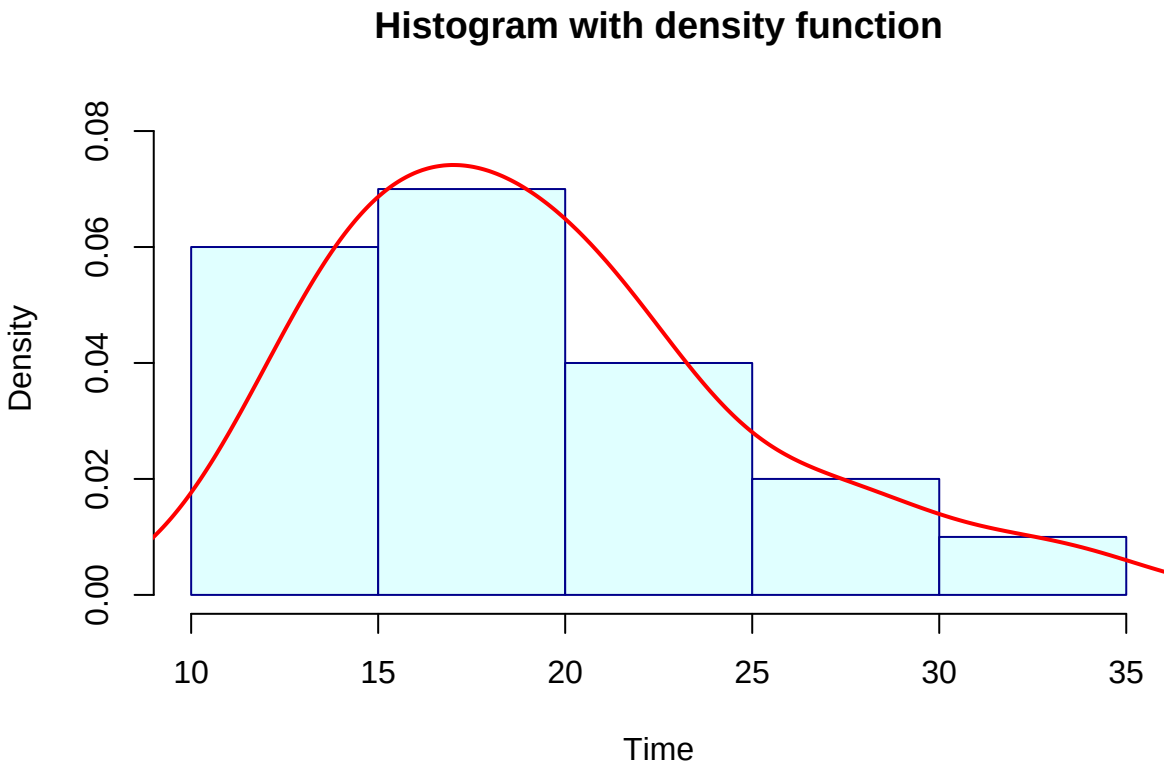
```
## [1] 0.844531
```

```r
# Visualize the skewness with a histogram and a density function
# First compute the density to determine the maximum height
dens <- density(Time)
y_max <- max(c(max(dens$y), max(hist(Time, freq = FALSE, plot = FALSE)$density)))
```

```
## Warning in hist.default(Time, freq = FALSE, plot = FALSE): argument 'freq' is
## not made use of
```

```r
# Create the histogram with adjusted y-limit
hist(Time,
     freq = FALSE,
     main = "Histogram with density function",
     xlab = "Time",
     col = "lightcyan",
     border = "darkblue",
     ylim = c(0, y_max * 1.1))  # 10% buffer above the maximum

# Add a density function
lines(dens, col = "red", lwd = 2)
```



Statistical explanation: Skewness is a measure of the asymmetry of a distribution. It describes whether

the distribution is symmetric or whether it has a longer tail on one side:

- **Positive skewness (> 0)**: The right tail is longer; most values lie to the left of the mean (right-skewed)
- **Negative skewness (< 0)**: The left tail is longer; most values lie to the right of the mean (left-skewed)
- **No skewness (0)**: The distribution is symmetric (as for a normal distribution)

Skewness is important for the choice of suitable statistical methods, since many procedures assume a symmetric distribution.

R explanation:

- The function `skewness()` from the `e1071` package computes the skewness of a numeric vector.
- The function `density()` estimates the probability density function of the data.
- With `lines()` a line can be added to an existing plot.
- The parameters `col` and `lwd` control the color and width of the line.

10 Filtering Datasets

Filters allow the selection of subsets of a dataset based on conditions.

```
library(dplyr)  # Load the dplyr package for data manipulation
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# Select the 'Price' column for restaurants of good quality
good_price <-
  Restaurant |>
  filter(Quality == 'Good') |>  # Filters rows where Quality equals 'Good'
  select(Price)                 # Selects only the Price column

print(good_price)
```

```
## # A tibble: 150 x 1
##    Price
##    <dbl>
##  1    22
##  2    33
##  3    19
##  4    11
##  5    23
##  6    33
##  7    44
##  8    30
##  9    33
## 10    22
## # i 140 more rows
```

```r
# Number of restaurants with good quality
num_good <- Restaurant |> filter(Quality == 'Good') |> nrow()
print(num_good)
```

```
## [1] 150
```

```r
# Number of restaurants with Price <= 20
num_cheap <- Restaurant |> filter(Price <= 20) |> nrow()
print(num_cheap)
```

```
## [1] 99
```

```r
# Number of restaurants with good quality AND Price <= 20
num_good_and_cheap <- Restaurant |> filter(Quality == 'Good' & Price <= 20) |> nrow()
print(num_good_and_cheap)
```

```
## [1] 43
```

```r
# Number of restaurants with good quality OR Price <= 20
num_good_or_cheap <- Restaurant |> filter(Quality == 'Good' | Price <= 20) |> nrow()
print(num_good_or_cheap)
```

```
## [1] 206
```

```r
# Example of more complex filtering with multiple conditions
complex_filter <- Restaurant |>
  filter((Quality == 'Good' | Quality == 'Excellent') & Price < 30) |>
  arrange(Price)  # Sort by price
head(complex_filter)
```

```
## # A tibble: 6 x 3
##   Restaurant Quality Price
##        <dbl> <chr>   <dbl>
## 1         53 Good       10
## 2          8 Good       11
## 3        169 Good       11
## 4        186 Good       11
## 5        188 Good       11
## 6         69 Good       12
```

Statistical explanation: Filtering data is a fundamental step in exploratory data analysis. It makes it possible to isolate specific subsets of a dataset and analyse them separately. This is particularly useful for:

- Testing hypotheses about subgroups
- Examining relationships between variables in specific segments
- Preparing data for particular analyses

R explanation:

- The package `dplyr` is part of the tidyverse collection and provides powerful functions for data manipulation.
- The function `filter()` selects rows based on conditions.
- Logical operators in R:
  - `==`: equality

  - `!=`: inequality
  - `<`, `<=`, `>`, `>=`: comparison operators
  - `&`: logical AND (both conditions must be satisfied)
  - `|`: logical OR (at least one condition must be satisfied)
  - `!`: logical negation
- The function `select()` selects columns.
- The function `arrange()` sorts the data by one or more columns.
- The function `nrow()` returns the number of rows.

11 Cross Tabulations

Cross tabulations (contingency tables) summarise the data for two variables in tabular form.

```
# Extract the Quality and Price variables from the Restaurant dataset
Quality <- Restaurant$Quality
Price <- Restaurant$Price

# Simple cross tabulation
ct1 <- table(Quality, Price)
print(ct1)
```

```
##                Price
## Quality         10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
##   Disappointing  6  4  3  3  2  4  4  5  5  6 10  1  3  5  1  3  4  5  5  3  0
##   Excellent      0  0  0  1  0  0  0  1  0  0  2  1  1  2  1  1  1  3  2  0  4
##   Good           1  4  3  5  6  1  5  3  3  3  9  9  4  6  5  9  4  8 10  0  7
##                Price
## Quality         31 32 33 34 35 36 37 38 40 41 42 43 44 45 46 47 48
##   Disappointing  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0
##   Excellent      3  5  3  2  3  4  2  2  4  4  2  1  2  2  3  2  2
##   Good           5  6  6  5  5  2  4  6  1  0  2  0  1  1  0  0  1
```

```
# Cross tabulation with grouped prices
bins <- 10 * (0:5)
Price_ranges <- Price |> cut(bins)
# Create a cross tabulation with the price ranges
ct2 <- table(Quality, Price_ranges)
print(ct2)
```

```
##                Price_ranges
## Quality         (0,10] (10,20] (20,30] (30,40] (40,50]
##   Disappointing      6      46      30       2       0
##   Excellent          0       4      16      28      18
##   Good               1      42      62      40       5
```

```
# Compute the relative frequencies (row-wise)
prop.table(ct2, margin = 1)  # margin = 1 means row-wise
```

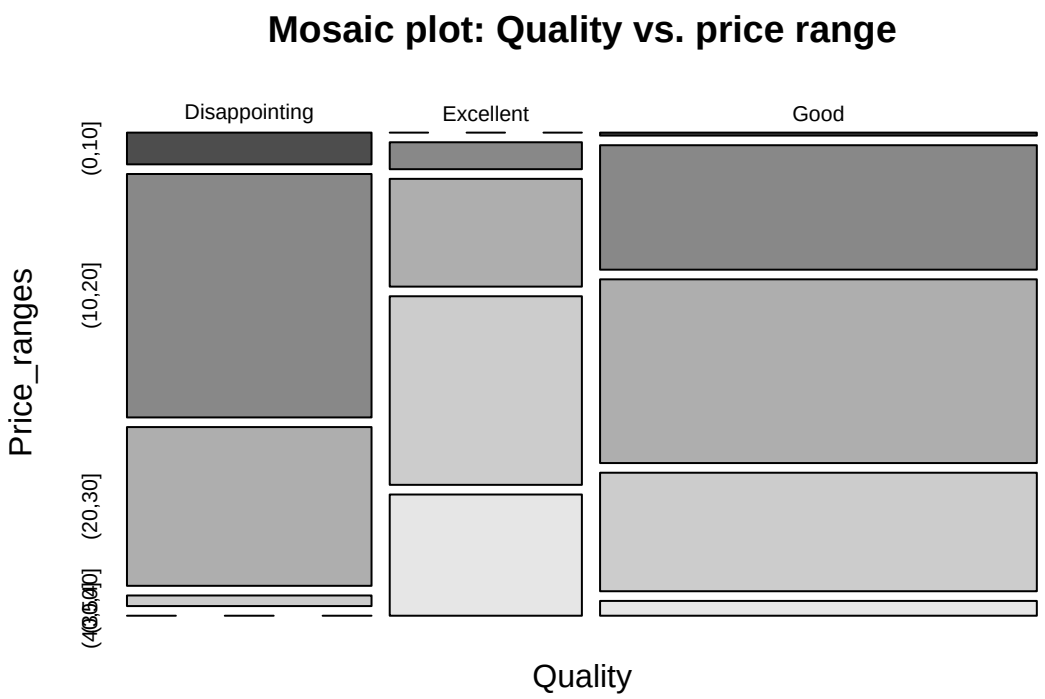
```
##                Price_ranges
## Quality              (0,10]     (10,20]     (20,30]     (30,40]     (40,50]
##   Disappointing 0.071428571 0.547619048 0.357142857 0.023809524 0.000000000
##   Excellent     0.000000000 0.060606061 0.242424242 0.424242424 0.272727273
```

```
##   Good          0.006666667 0.280000000 0.413333333 0.266666667 0.033333333
```

```
# Compute the relative frequencies (column-wise)
prop.table(ct2, margin = 2)  # margin = 2 means column-wise
```

```
##                Price_ranges
## Quality             (0,10]    (10,20]    (20,30]    (30,40]    (40,50]
##   Disappointing 0.85714286 0.50000000 0.27777778 0.02857143 0.00000000
##   Excellent     0.00000000 0.04347826 0.14814815 0.40000000 0.78260870
##   Good          0.14285714 0.45652174 0.57407407 0.57142857 0.21739130
```

```
# Visualise the cross tabulation as a mosaic plot
mosaicplot(ct2,
           main = "Mosaic plot: Quality vs. price range",
           color = TRUE)
```



Statistical explanation: Cross tabulations (also called contingency tables or two-way frequency tables) show the joint frequency distribution of two categorical variables. They are an important tool for examining relationships between categorical variables:

- The rows represent the categories of the first variable
- The columns represent the categories of the second variable
- Each cell contains the number of observations belonging to both categories

Cross tabulations can be used to:

- Identify associations between categorical variables
- Analyse conditional distributions
- Carry out chi-squared tests of independence

R explanation:

- The function `table()` creates a cross tabulation from two or more vectors.

- With `prop.table()` relative frequencies can be computed:
  - `margin = 1`: row-wise relative frequencies (sum of each row = 1)
  - `margin = 2`: column-wise relative frequencies (sum of each column = 1)
  - Without `margin`: overall relative frequencies (sum of all cells = 1)
- The function `mosaicplot()` creates a graphical representation of a cross tabulation in which the areas are proportional to the frequencies.

12 Recoding and Reshaping

In recoding, variables are recoded or restructured.

```
library(stringr)  # Package for string manipulation

# Split a variable
shirt_split <- Inventory |>
  pull(shirt) |>                # Extracts the shirt column as a vector
  str_split_fixed(',', 3)       # Splits each string at commas into 3 parts

# Show the first rows of the result
head(shirt_split)
```

```
##      [,1]         [,2]    [,3]
## [1,] "T-shirt"    "blue"  "S"
## [2,] "T-shirt"    "white" "M"
## [3,] "T-shirt"    "white" "S"
## [4,] "Polo shirt" "blue"  "XS"
## [5,] "T-shirt"    "green" "L"
## [6,] "T-shirt"    "red"   "XS"
```

```
# Create a new data frame
Inventory2 <- shirt_split |>
  data.frame() |>               # Convert the matrix into a data frame
  setNames(c('style', 'colour', 'size'))  # Name the columns

# Add the prices as a new column
Inventory2$price <- Inventory |> pull(price)

# Compute a 30% discount on the price and add it as a new column 'discount'
Inventory2$discount <- Inventory2$price * (1 - 0.30)

# Inspect the structure of the new data frame
str(Inventory2)
```

```
## 'data.frame':    200 obs. of  5 variables:
##  $ style   : chr  "T-shirt" "T-shirt" "T-shirt" "Polo shirt" ...
##  $ colour  : chr  "blue" "white" "white" "blue" ...
##  $ size    : chr  "S" "M" "S" "XS" ...
##  $ price   : num  75 115 105 75 100 70 85 75 105 115 ...
##  $ discount: num  52.5 80.5 73.5 52.5 70 49 59.5 52.5 73.5 80.5 ...
```

```
# Example for recoding a categorical variable
# Create a new column 'price_category' based on the price
```

```r
Inventory2$price_category <- cut(Inventory2$price,
                                 breaks = c(0, 20, 40, 60, Inf),
                                 labels = c("Cheap", "Medium", "Expensive", "Premium"))

# Show the frequencies of the price categories
table(Inventory2$price_category)
```

```
##
##     Cheap    Medium Expensive   Premium
##         0         0         0       200
```

Statistical explanation: Recoding is an important step in data preparation in which variables are transformed or restructured. This can serve various purposes:

- **Splitting variables**: A variable that contains multiple pieces of information is split into separate variables
- **Categorisation**: Continuous variables are converted into categories
- **Recoding**: Changing values or labels for clearer interpretation
- **Computing derived variables**: Creating new variables based on existing ones

These transformations are often necessary to prepare data for specific analyses or to facilitate interpretation.

R explanation:

- The package `stringr` provides powerful functions for manipulating strings.
- The function `pull()` from `dplyr` extracts a column as a vector.
- `str_split_fixed()` splits strings at a specified separator and returns a matrix with a fixed number of columns.
- `data.frame()` converts a matrix into a data frame.
- `setNames()` renames the columns of a data frame.
- The `$` operator is used to access columns or to create new columns.

13 Aggregation

Aggregation summarises data.

```r
# Aggregate by the 'colour' variable
agg_result <- Inventory2 |>
  aggregate(list(Inventory2$colour), length)
print(agg_result)
```

```
##   Group.1 style colour size price discount price_category
## 1   black    44     44   44    44       44             44
## 2    blue    44     44   44    44       44             44
## 3   green    38     38   38    38       38             38
## 4     red    36     36   36    36       36             36
## 5   white    38     38   38    38       38             38
```

```r
# Compute the average price per colour
avg_price_by_color <- aggregate(price ~ colour, data = Inventory2, FUN = mean)
print(avg_price_by_color)
```

```
##   colour     price
## 1  black  98.18182
```

```
## 2   blue  98.18182
## 3  green 108.68421
## 4    red 100.69444
## 5  white 101.71053
```

```
# Multiple aggregations at once with dplyr
library(dplyr)
summary_by_color <- Inventory2 |>
  group_by(colour) |>
  summarise(
    count = n(),
    avg_price = mean(price),
    min_price = min(price),
    max_price = max(price),
    total_value = sum(price)
  )
print(summary_by_color)
```

```
## # A tibble: 5 x 6
##   colour count avg_price min_price max_price total_value
##   <chr>  <int>     <dbl>     <dbl>     <dbl>       <dbl>
## 1 black     44      98.2        70       135        4320
## 2 blue      44      98.2        70       135        4320
## 3 green     38     109.         75       135        4130
## 4 red       36     101.         70       135        3625
## 5 white     38     102.         70       130        3865
```

Statistical explanation: Aggregation is the process of grouping and summarising data in order to identify patterns and trends at a higher level. Typical aggregation functions are:

- **Count**: number of observations in each group
- **Sum**: total sum of values in each group
- **Mean**: average value in each group
- **Median**: middle value in each group
- **Minimum/Maximum**: smallest/largest value in each group
- **Standard deviation**: measure of the spread in each group

Aggregation is a central component of descriptive statistics and exploratory data analysis, since it reduces complex datasets to interpretable summary statistics.

R explanation:

- The function `aggregate()` in base R groups data and applies a function to each group.
- The formula notation `price ~ colour` means "aggregate price by colour".
- The package `dplyr` offers a more intuitive and flexible method for aggregation with `group_by()` and `summarise()`.
- The function `n()` in `dplyr` counts the number of observations in each group.
- With `summarise()` multiple aggregations can be performed at the same time.

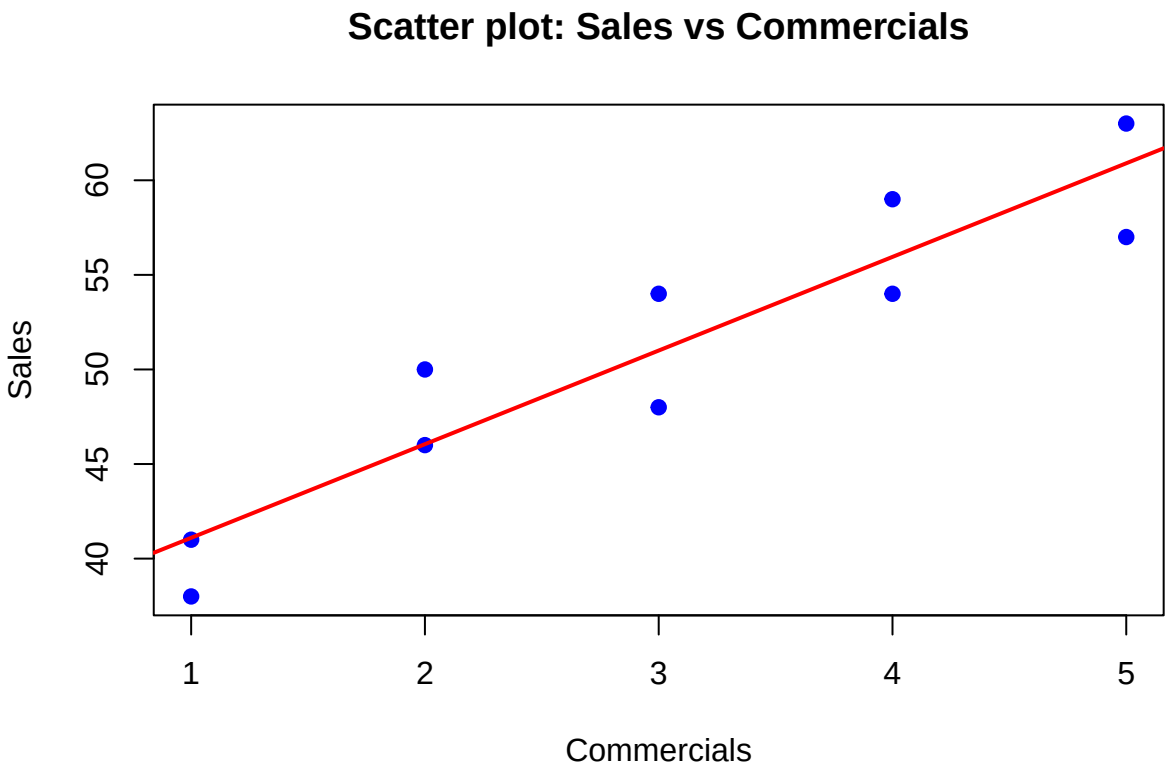
14 Scatter Plots

Scatter plots visualize the relationship between two quantitative variables.

```
# Create a scatter plot
plot(Sales ~ Commercials,
```

```
     data = Stereo,
     main = "Scatter plot: Sales vs Commercials",
     xlab = "Commercials",
     ylab = "Sales",
     pch = 19,                    # Point symbol (filled circle)
     col = "blue")

# Add a trend line via linear regression
fit <- lm(Sales ~ Commercials, data = Stereo)  # Linear regression model
abline(fit, col = "red", lwd = 2)              # Add the regression line
```




```
# Show the model summary
summary(fit)
```

```
##
## Call:
## lm(formula = Sales ~ Commercials, data = Stereo)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -3.900 -2.737 -0.075  2.775  3.950
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   36.150      2.285  15.820 2.55e-07 ***
## Commercials    4.950      0.689   7.185 9.39e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 3.081 on 8 degrees of freedom
## Multiple R-squared:  0.8658, Adjusted R-squared:  0.849
## F-statistic: 51.62 on 1 and 8 DF,  p-value: 9.386e-05
```

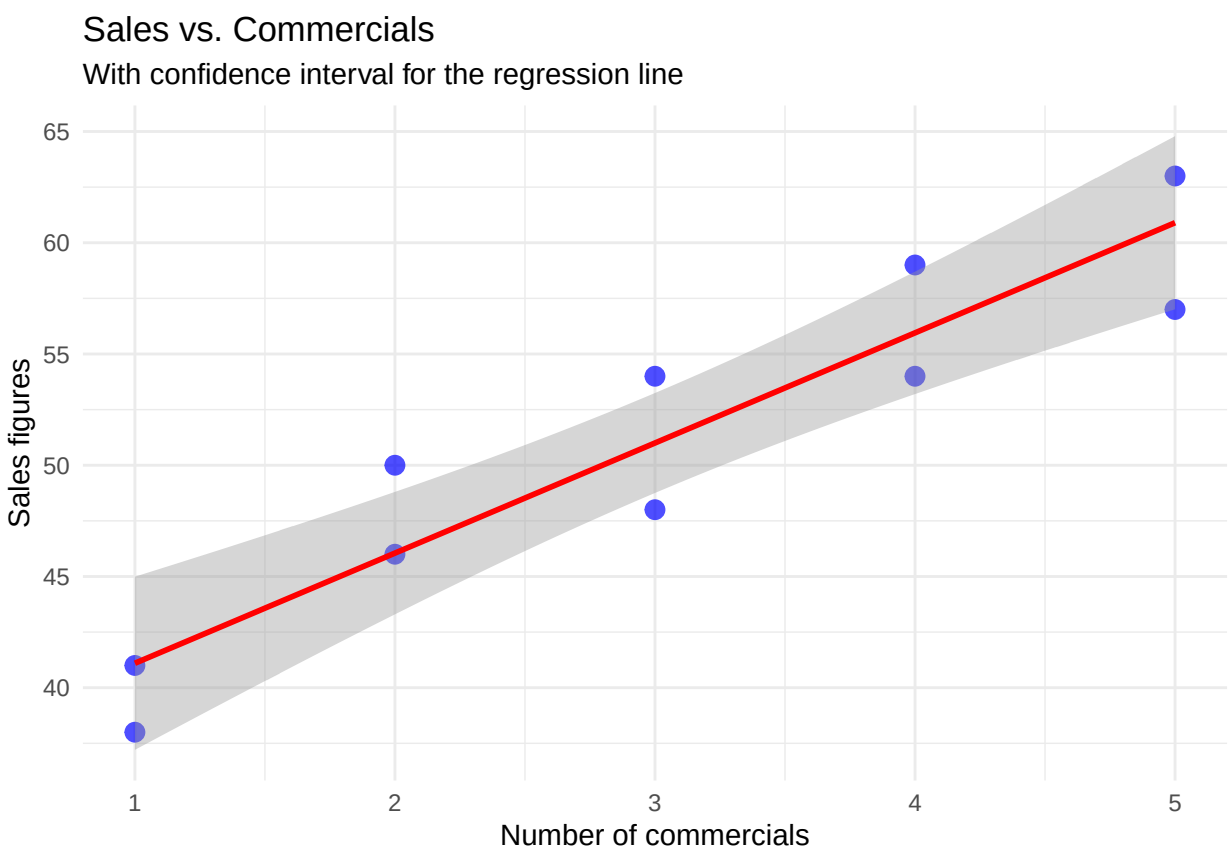
```
# Enhanced scatter plot with ggplot2
library(ggplot2)
```

```
##
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:e1071':
##
##     element
```

```
ggplot(Stereo, aes(x = Commercials, y = Sales)) +
  geom_point(color = "blue", size = 3, alpha = 0.7) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Sales vs. Commercials",
       subtitle = "With confidence interval for the regression line",
       x = "Number of commercials",
       y = "Sales figures") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Statistical explanation: Scatter plots are a fundamental method for visualising the relationship between two continuous variables. Each point in the plot represents an observation, with the position of the point determined by the values of the two variables.

Scatter plots help to:

- Recognize the type of relationship (linear, non-linear)
- Assess the strength of the relationship
- Identify patterns, clusters, and outliers

Linear regression is a statistical method for modelling the linear relationship between two variables. The regression line minimises the sum of the squared vertical distances between the data points and the line.

R explanation:

- The function `plot()` with the formula notation `y ~ x` creates a scatter plot.
- Parameters for `plot()`:
  - `pch`: plot character (symbol for the points)
  - `col`: color of the points
  - `main`, `xlab`, `ylab`: title and axis labels
- The function `lm()` (linear model) fits a linear regression.
- The function `abline()` adds a line to an existing plot.
- The function `summary()` outputs a detailed summary of the regression model.
- The package `ggplot2` provides an alternative, powerful method for creating graphics.

Note on interpretation: The slope of the trend line provides information about the type of relationship:

- Positive slope: positive relationship (when x increases, y also increases)
- No slope: no relationship
- Negative slope: negative relationship (when x increases, y decreases)

The R^2 value (reported in `summary(fit)` as "Multiple R-squared") indicates the proportion of variance in the dependent variable that is explained by the model. A higher value indicates a stronger linear relationship.